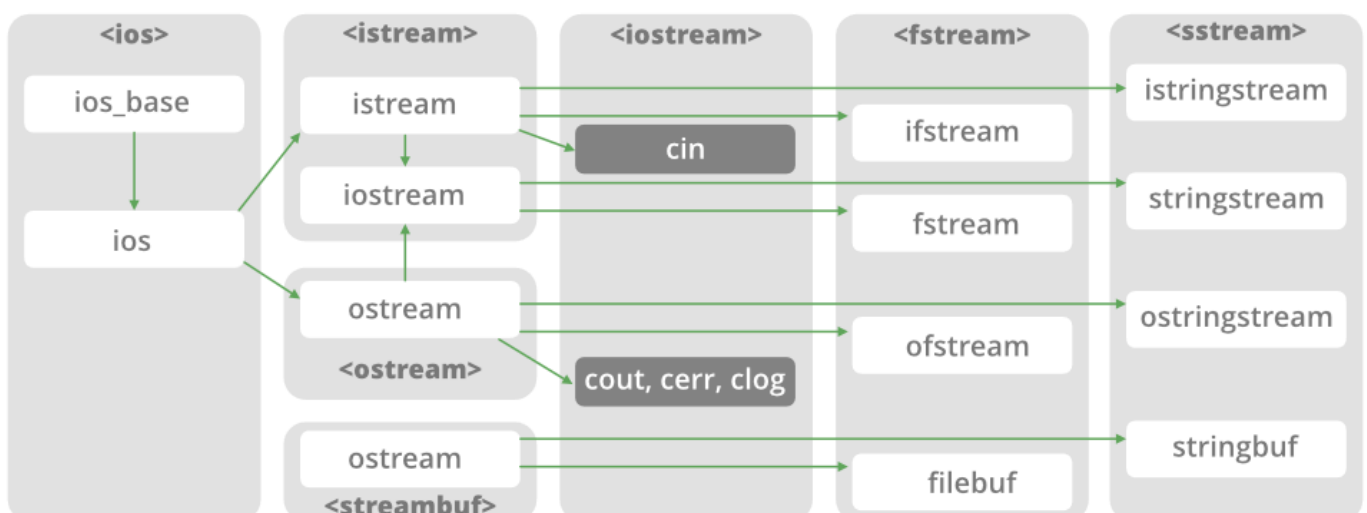


Seminar

File streams. Text files.

1. File streams.

- Relative vs absolute path
 - Relative path - path from the current directory
 - Absolute path
 - Note: Windows's paths use '`\`' and to write that in C++ we use '`\\`'
- File streams, just like the IO streams - `cout` and `cin`, are streams for input and output of information, but to a certain **file**.
- `std::ifstream`, `std::ofstream` and `std::fstream` are *classes* and we'll be creating objects from these classes to interact with files.
Note: **`cin`** is an object of type `istream`, and **`cout`**, **`cerr`** and **`clog`** - `ostream`
- Text files
 - Used for storing text (usually .txt)
 - Text editors can view the information in the file
 - Easy IO with `>>`, `<<` and `getline` (*just like `cin` and `cout`*)
- Binary files
 - Used for storing objects (usually .dat)
 - Text editors won't display the file correctly
 - For IO read and write methods must be used
- Steps when working with files
 1. Ask for a file to be opened.
 2. Check if the file has been opened successfully.
 3. Work with the file.
 4. Close the file as soon as we are done with it.



- File stream creation and methods, *usually used for text files*
 - `std::[i/o]fstream <identifier>;`
 - `open(<file_path>, [flags])` - open file for reading or writing.
 - `close()` - close the file.
 - `eof()` - returns true when the end of the file is reached.
 - `bad()` - returns true if a reading or writing operation fails.
 - `is_open()` - returns true if the file is successfully opened.
 - `gcount()` - returns the number of bytes read thus far.
 - `ignore(<num>)` - skips <num> bytes from the file.
 - `peek()` - checks the next available character.

Notes:

By default when opening:

- files for **writing**, by default *the content of the file is erased*.
- a **non-existing** file for reading, that file *will be created*.

- Flags when opening files
 - Additional options can be added when opening files that are defined in the second parameter of the open method (or in the constructor).
 - Flags:
 - `std::ios::app` - when writing, **append** at the end of the file.
 - `std::ios::trunc` - when a file has been opened for writing, erase (truncate) its content first before writing.
 - `std::ios::binary` - open a file in binary mode.
 - `std::ios::in` - open file for reading (*when an fstream object is used*)
 - `std::ios::out` - open file for writing (*when an fstream object is used*)
 - Note: Multiple flags can be added using the binary OR operator (`|`)
- Example:
- ```
std::fstream file("myFile.bin", std::ios::in | std::ios::binary);
```

- Simple text file example:

```
std::ofstream out("file.txt"); // Step 1
if (!out) { // Step 2
 std::cout << "Couldn't open file for writing!" << std::endl;
 return 0;
}

std::out << "This text will go in the file" << std::endl; // Step 3
out.close(); // Step 4
```